

INVENTARIUM

Sistema de Gestión de Almacén — Taller

Documentación Técnica del Módulo Web

Proyecto	Gestión Taller — Equipo1
Versión	1.0
Tecnologías usadas	Node.js · MySQL · HTML/CSS/JS · Apache
Entorno	Ubuntu Server + Apache (proxy) + Node.js :3000
Fecha	Mayo 2026

1. Introducción

INVENTARIUM es la web de gestión de almacén integrado en el proyecto Gestión Taller. Permite al personal del taller consultar el contenido físico de cada hueco del armario de materiales a través de un navegador.

La web forma parte de un sistema más amplio que incluye base de datos MySQL compartida, autenticación de usuarios y otras secciones aún en desarrollo. Esta documentación cubre exclusivamente la capa web: servidor Node.js, páginas HTML, estilos CSS y lógica JavaScript.

1.1 Alcance de este documento

- Arquitectura del servidor Node.js y su integración con Apache.
- Descripción de cada fichero fuente de la web.
- Flujo de autenticación y consulta de datos.
- Estructura de la base de datos (tablas implicadas).

2. Arquitectura General

El sistema sigue una arquitectura de tres capas clásica adaptada al entorno del taller:

Capa	Tecnología	Descripción
Presentación	HTML · CSS · JavaScript (Vanilla)	Interfaz de usuario servida como ficheros estáticos.
Aplicación	Node.js (http nativo)	Servidor que gestiona rutas, autenticación y servicio de estáticos.
Datos	MySQL 8+	Base de datos Gestión Taller compartida con el resto del proyecto.

2.1 Rol de Apache como proxy inverso

En el servidor Ubuntu, Apache actúa como proxy inverso: recibe las peticiones HTTP/HTTPS en los puertos 80/443 y las reenvía al proceso Node.js que escucha internamente en el puerto 3000. Esto permite:

- Aprovechar el control de acceso y los logs de Apache.
- Convivir con otros Virtual Hosts o aplicaciones en el mismo servidor.

3. Estructura de Ficheros

Todos los ficheros de la web residen en el mismo directorio de trabajo de Node.js.

```
/proyecto-web/
├── server.js           ← Servidor Node.js principal
├── package.json        ← Dependencias npm
├── index.html          ← Pantalla de login
├── armario.html        ← Vista principal del armario
├── pendiente.html      ← Sección en desarrollo
├── 404.html            ← Página de error personalizada
├── styles.css          ← Hoja de estilos global
├── main.js            ← Lógica de la web
├── src/
│   └── error404.png    ← Imagen para la página 404
```

Fichero	Tipo	Función principal
server.js	Node.js	Servidor HTTP, API REST /api, login, servicio de estáticos y página 404.
package.json	JSON	Define el proyecto npm y la dependencia mysql2.
index.html	HTML	Formulario de login. Envía POST a /api/login.
armario.html	HTML	Mapa visual del armario con 22 celdas/huecos clicables.
pendiente.html	HTML	Página de marcador para secciones futuras.
404.html	HTML	Respuesta 404 personalizada con imagen del robot.
styles.css	CSS	Estilos globales: tema oscuro, layout armario, panel lateral
main.js	JS	Gestión de clicks en celdas, fetch a /api y renderizado de la tabla de material.

4. server.js — Servidor Node.js

El servidor está implementado usando únicamente módulos nativos de Node.js (http, fs, path, url) más la dependencia mysql2.

4.1 Configuración

El objeto CONFIG centraliza los parámetros del entorno:

```
const CONFIG = {  
  PORT: 3000,  
  DB_HOST: "192.168.56.10",  
  DB_PORT: 3306,  
  DB_USER: "inventario",  
  DB_PASS: "LanciaYpsilonRally2HFIntegrale",  
  DB_NAME: "gestion_taller",  
};
```

4.2 Rutas expuestas

Método	Ruta	Auth	Descripción
GET	/	No	Sirve index.html (login).
POST	/api/login	No	Valida usuario/contraseña contra la tabla usuarios. Redirige a /armario.html o /?error=1.
GET	/api	No	Recibe ?codigo=N, consulta MySQL y devuelve JSON con el material de esa ubicación.
GET	/*	No	Sirve cualquier fichero estático con MIME type correcto. Devuelve 404.html si no existe.

4.3 Seguridad implementada

- MIME filtering: solo se sirven extensiones registradas en MIME_TYPES; cualquier otra devuelve 404.
- Fichero .env: El proyecto incluye un fichero .env en la raíz del directorio de trabajo. Este fichero centraliza todas las variables de configuración sensibles, evitando que aparezcan hardcodeadas en el código fuente.

5. Base de Datos

La base de datos Gestion_Taller es compartida con el resto del proyecto. El módulo web solo realiza lecturas (SELECT) sobre las tablas descritas a continuación.

5.1 Tablas implicadas

Tabla	Clave primaria	Descripción
material	id (implícito)	Registro de cada pieza/herramienta. Contiene nombre, descripción, cantidad, observaciones y FKs a ubicacion, estado y subcategoria.
ubicaciones	codigo_armario	Mapea el código numérico del hueco (1-22) con su etiqueta textual.
estado	id_estado	Catálogo de estados posibles: Disponible, En uso, Averiado, etc.
categorias	id_categoria	Clasificación del material (ej. Herramienta, Consumible).
subcategorias	id_subcategoria	Subclasificación del material relacionada a una categoría.
usuarios	id	Credenciales de acceso (nombre + contraseña). Solo la lectura de login usa esta tabla.

5.2 Consulta principal

La función getMaterial ejecuta el siguiente SELECT para obtener el material de un hueco:

```
SELECT
  m.nombre, m.descripcion, m.cantidad, m.observaciones,
  e.nombre AS estado,
  c.nombre AS categoria,
  s.nombre AS subcategoria
FROM material m
JOIN ubicaciones u ON m.id_ubicacion = u.codigo_armario
JOIN estado e ON m.id_estado = e.id_estado
JOIN subcategorias s ON m.id_subcategoria = s.id_subcategoria
JOIN categorias c ON s.id_categoria = c.id_categoria
WHERE u.codigo_armario = ?
ORDER BY c.nombre, s.nombre, m.nombre
```

6. Parte visual de la web

6.1 index.html — Login

Presenta el formulario de acceso con los campos usuario y contraseña. Al enviar, realiza un POST a `/api/login`. En caso de error muestra un mensaje.

6.2 armario.html — Vista principal

Contiene el mapa visual del armario con 22 celdas distribuidas en:

- Bloque superior: 8 celdas en 2 filas (A1–A4, B1–B4).
- Bloque central: puerta Taller (9), zona interior con 9 estantes (C1–E3) y 3 armarios bajos (F1–F3), puerta DTD (22).

6.3 main.js — Lógica de interacción

El fichero `main.js` gestiona el ciclo completo de interacción:

- Registra listeners de clic en todas las celdas `.celda`.
- Gestiona el estado de celda seleccionada (marca con clase CSS `.selected`).
- Llama a `/api?codigo=N` via Fetch API y maneja los estados: cargando, error, tabla vacía y tabla con datos.
- Renderiza los datos en una tabla HTML dinámica con badges de estado (badge-Disponible, badge-En-uso, etc.).
- Cierra el panel con el botón “X” o la tecla Escape.

6.4 styles.css — Sistema de estilos

La hoja de estilos implementa un tema oscuro coherente con la identidad visual del proyecto. Los puntos destacados son:

- Variables CSS para colores y espaciados consistentes.
- Layout del armario mediante CSS
- Panel lateral con transición mediante `transform` y `transition`.
- Badges de estado con colores (verde = disponible, rojo = averiado, etc.).
- Responsive: columnas del armario se adaptan en pantallas pequeñas.

7. Flujos Principales

7.1 Flujo de autenticación

#	Actor	Acción
1	Usuario	Abre / en el navegador → Apache reenvía a Node.js → se devuelve index.html.
2	Usuario	Introduce credenciales y envía el formulario (POST /api/login).
3	Node.js	Consulta SELECT en tabla usuarios. Si coincide → redirect 302 a /armario.html.
4	Node.js	Si no coincide → redirect 302 a /?error=1. El JS del cliente muestra el mensaje de error.

7.2 Flujo de consulta de ubicación

#	Actor	Acción
1	Usuario	Hace clic sobre una celda del armario (ej. "C2").
2	main.js	Abre el panel lateral
3	Node.js	Valida el parámetro, ejecuta la query con JOIN y devuelve array JSON.
4	main.js	Si data.length > 0: renderiza la tabla. Si no hay material: muestra mensaje "sin registros".

10.1 Contenido y descripción de cada variable

Variable	Valor	Tipo	Descripción
PORT	3000	Numero	Puerto TCP en el que Node.js escucha.
DB_HOST	192.168.56.10	IP privada	Dirección IP del servidor MySQL. Al no ser localhost indica que la BD corre en una máquina separada (o VM) dentro de la red interna 192.168.56.0/24
DB_PORT	3306	Numero	Puerto estándar de MySQL.
DB_USER	inventario	Cadena	Usuario de MySQL con permisos de SELECT sobre las tablas del módulo y de lectura en usuarios para el login.
DB_PASS	***** (redactado)	Secreto	Contraseña del usuario inventario. Se usa una frase de contraseña larga.
DB_NAME	gestion_taller	Cadena	Nombre de la base de datos.

10.2 Seguridad del fichero .env

- Añadir .env al fichero .gitignore para que nunca se suba al repositorio.
- Restringir permisos en el servidor: `chmod 600 .env` (solo lectura por el propietario del proceso Node.js).
- Rotar la contraseña DB_PASS periódicamente y actualizar el fichero .env en todos los entornos.